

Module 2: Variables, Constants, and Data Types in PL/SQL

Learning Outcomes

At the end of this module, students will be able to:

- Understand the purpose of variables in PL/SQL.
 - Declare and initialize variables.
 - Differentiate between variables and constants.
 - Use different PL/SQL data types.
 - Understand the use of %TYPE and %ROWTYPE.
 - Understand the scope and lifetime of variables.
 - Write PL/SQL programs using variables.
-

2.1 Introduction to Variables

What is a Variable?

A **variable** is a named memory location used to temporarily store data during the execution of a PL/SQL program. Unlike SQL, which works directly with database tables, PL/SQL allows developers to store intermediate results in variables and manipulate them before updating the database.

Definition

A variable is an identifier that stores a value which can change during program execution.

Why Do We Need Variables?

Suppose you want to calculate the annual salary of an employee.

Without variables:

- The calculation must be repeated multiple times.
- Intermediate results cannot be stored.
- Programs become lengthy and difficult to understand.

Variables help us:

- Store temporary values.
 - Perform calculations.
 - Improve readability.
 - Reduce repeated calculations.
-

Real-Life Example

Imagine calculating a student's percentage.

Student Marks

Subject Marks

DBMS 85

Java 90

Python 88

Instead of directly writing calculations multiple times, we store them in variables.

marks1 = 85

marks2 = 90

marks3 = 88

total = marks1 + marks2 + marks3

percentage = total / 3

PL/SQL works in the same manner.

Variable Declaration

Variables are declared in the **DECLARE** section.

Syntax

DECLARE

variable_name datatype;

BEGIN

executable statements;

```
END;  
/
```

Example 1

```
DECLARE  
    v_name VARCHAR2(50);  
BEGIN  
    v_name := 'Rahul';  
  
    DBMS_OUTPUT.PUT_LINE(v_name);  
END;  
/
```

Output

Rahul

Example 2

```
DECLARE  
    v_age NUMBER;  
BEGIN  
    v_age := 20;  
  
    DBMS_OUTPUT.PUT_LINE('Age = ' || v_age);  
END;  
/
```

Output

Age = 20

Variable Naming Rules

A variable name:

- Must begin with a letter.
- Can contain numbers.
- Can contain underscore (_).
- Cannot contain spaces.
- Cannot use reserved keywords.
- Maximum length is 30 characters.

Valid Variable Names

v_salary
student_name
emp101
total_marks

Invalid Variable Names

1salary
student name
select
employee-id

Assignment Operator

PL/SQL uses

:=

to assign values.

Example

salary := 50000;

Do not confuse it with SQL.

SQL uses

=

PL/SQL uses

:=

2.2 Data Types

What is a Data Type?

A data type specifies the type of value that can be stored in a variable.

Examples

- Integer
- Decimal
- Character
- Date
- Boolean

Categories of Data Types

PL/SQL supports

- Scalar Data Types
- Composite Data Types
- Reference Data Types
- Large Object (LOB) Data Types

For beginners, we focus on **Scalar Data Types**.

NUMBER

Stores numeric values.

Example

DECLARE

 v_marks NUMBER;

BEGIN

 v_marks := 95;

 DBMS_OUTPUT.PUT_LINE(v_marks);

END;

/

Output

95

VARCHAR2

Stores variable-length strings.

DECLARE

 v_city VARCHAR2(20);

BEGIN

 v_city := 'Delhi';

 DBMS_OUTPUT.PUT_LINE(v_city);

END;

/

Output

Delhi

CHAR

Stores fixed-length strings.

DECLARE

 grade CHAR(1);

BEGIN

 grade := 'A';

```
    DBMS_OUTPUT.PUT_LINE(grade);
END;
/
Output
A
```

DATE

Stores dates.

```
DECLARE
    joining_date DATE;
BEGIN
    joining_date := SYSDATE;

    DBMS_OUTPUT.PUT_LINE(joining_date);
END;
/
Output
05-JUL-26
```

BOOLEAN

Stores TRUE or FALSE.

```
DECLARE
    is_pass BOOLEAN;
BEGIN
    is_pass := TRUE;

    IF is_pass THEN
        DBMS_OUTPUT.PUT_LINE('Student Passed');
    END IF;
END;
/
Output
Student Passed
```

Common Data Types

Data Type	Description	Example
NUMBER	Numeric values	100
VARCHAR2	Variable-length text	Oracle
CHAR	Fixed-length text	A
DATE	Date values	SYSDATE
BOOLEAN	Logical values	TRUE

2.3 Constants

What is a Constant?

A constant is a variable whose value **cannot be changed** after initialization.

Syntax

```
constant_name CONSTANT datatype := value;
```

Example

```
DECLARE
    pi CONSTANT NUMBER := 3.14159;
BEGIN
    DBMS_OUTPUT.PUT_LINE(pi);
END;
```

/

Output
3.14159

Attempting to modify pi will result in a compilation error.

Variables vs Constants

Variable	Constant
Value can change	Value cannot change
Uses :=	Uses CONSTANT keyword
Used for temporary data	Used for fixed values

2.4 Variable Initialization

Variables may be initialized while declaring them.

```
DECLARE
    salary NUMBER := 40000;
BEGIN
    DBMS_OUTPUT.PUT_LINE(salary);
END;
```

/

Multiple Variables

```
DECLARE
    first_name VARCHAR2(30);
    last_name VARCHAR2(30);
BEGIN
    first_name := 'Komal';
    last_name := 'Pundir';

    DBMS_OUTPUT.PUT_LINE(first_name || ' ' || last_name);
END;
```

/

Output
Komal Pundir

2.5 %TYPE Attribute

What is %TYPE?

Suppose the salary column in the EMPLOYEE table changes from NUMBER(8) to NUMBER(12).

If your PL/SQL program declares:

```
salary NUMBER(8);
```

you must manually update every program.

To avoid this, Oracle introduced %TYPE.

It automatically adopts the data type of an existing table column.

Employee Table

EMP_ID	EMP_NAME	SALARY
--------	----------	--------

101	Rahul	45000
-----	-------	-------

Example

```
DECLARE
    v_salary employee.salary%TYPE;
BEGIN
    SELECT salary
    INTO v_salary
    FROM employee
```

```
WHERE emp_id = 101;

DBMS_OUTPUT.PUT_LINE(v_salary);
END;
/
```

Advantages

- Reduces maintenance.
- Prevents datatype mismatch.
- Improves reliability.

2.6 %ROWTYPE

Sometimes we need an entire row instead of a single column.

%ROWTYPE creates a record with the same structure as a database table.

Example

```
DECLARE
    emp_record employee%ROWTYPE;
BEGIN
    SELECT *
    INTO emp_record
    FROM employee
    WHERE emp_id = 101;

    DBMS_OUTPUT.PUT_LINE(emp_record.emp_name);
    DBMS_OUTPUT.PUT_LINE(emp_record.salary);
END;
/
```

%TYPE vs %ROWTYPE

%TYPE	%ROWTYPE
Refers to one column	Refers to an entire row
Stores one value	Stores multiple values
Uses less memory	Uses more memory
Most commonly used	Used when all column values are needed

2.7 Scope of Variables

The **scope** of a variable is the region of the program where it can be accessed.

Example

```
DECLARE
    x NUMBER := 10;
BEGIN
    DBMS_OUTPUT.PUT_LINE(x);
END;
/
```

Here, x is available only within this PL/SQL block.

Complete Example

```
DECLARE
    student_name VARCHAR2(50);
    marks NUMBER;
    percentage NUMBER;
BEGIN
    student_name := 'Rohit';
    marks := 450;
```

```
percentage := marks / 5;

DBMS_OUTPUT.PUT_LINE('Student : ' || student_name);
DBMS_OUTPUT.PUT_LINE('Marks : ' || marks);
DBMS_OUTPUT.PUT_LINE('Percentage : ' || percentage || '%');
END;
/
Output
Student : Rohit
Marks : 450
Percentage : 90%
```

Real-World Example: Employee Salary

```
DECLARE
    basic_salary NUMBER := 40000;
    hra NUMBER;
    da NUMBER;
    gross_salary NUMBER;
BEGIN
    hra := basic_salary * 0.20;
    da := basic_salary * 0.10;

    gross_salary := basic_salary + hra + da;

    DBMS_OUTPUT.PUT_LINE('Basic Salary : ' || basic_salary);
    DBMS_OUTPUT.PUT_LINE('HRA : ' || hra);
    DBMS_OUTPUT.PUT_LINE('DA : ' || da);
    DBMS_OUTPUT.PUT_LINE('Gross Salary : ' || gross_salary);
END;
/
```

Summary

In this module, we explored variables, constants, and data types in PL/SQL. We learned how to declare and initialize variables, use Oracle's built-in data types, and leverage %TYPE and %ROWTYPE to create maintainable and reliable programs. These concepts provide the foundation for writing PL/SQL blocks that perform calculations and interact effectively with database tables.

Laboratory Exercises

1. Write a PL/SQL program to display your name, roll number, and branch using variables.
 2. Calculate the area of a rectangle using variables for length and breadth.
 3. Write a program to calculate the simple interest using variables.
 4. Create a constant for the value of PI and calculate the area of a circle.
 5. Display the current system date using a DATE variable.
 6. Fetch an employee's salary using %TYPE.
 7. Fetch and display all details of an employee using %ROWTYPE.
 8. Calculate the gross salary using Basic Salary, HRA, and DA.
 9. Demonstrate the scope of variables using nested PL/SQL blocks.
 10. Write a program to concatenate a student's first name and last name using VARCHAR2.
-

Viva/Interview Questions

1. What is a variable in PL/SQL?
2. Explain the purpose of the DECLARE section.
3. Differentiate between NUMBER, VARCHAR2, CHAR, and DATE.
4. What is the difference between a variable and a constant?
5. What is the assignment operator in PL/SQL?

6. Why should %TYPE be preferred over explicitly declaring a data type?
7. What is %ROWTYPE, and when is it useful?
8. What is the scope of a variable in PL/SQL?
9. Can a constant be modified after initialization? Why or why not?
10. What are the advantages of using Oracle's anchored data types (%TYPE and %ROWTYPE) in enterprise applications?